

Final Project Report

Bryce Poonai

Introduction

Ever since the introduction of Deep Learning models, Computer Vision has drastically improved in both performance and versatility. Most Computer Vision models are already pre-trained for object detection and shape/pattern recognition, so they can be fine-tuned for any application. Using the latest Computer Vision models, what is the optimal training and testing amount for a niche application that would give us sufficient performance without using large datasets and time. In this case, we will use circuit models.

Literature Survey

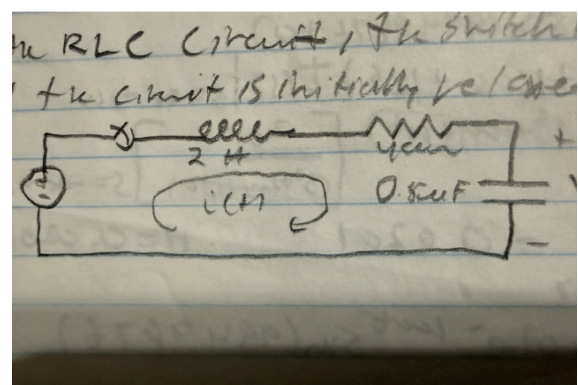
Deep Learning models have shown drastic increments in performance over the past couple years because of advancements in the hardware used for training the models and improvements to the Deep Learning models themselves, so it's interesting to see how these models would perform in many different applications using hardware anyone can use within reasonable amounts of time. Typical good performance of a model is if it manages to capture all the necessary objects in the picture with high confidence scores¹. While we can determine the model's performance on a set test image and look at how it performs using this metric, it doesn't give us the full picture of how the model will perform on other samples of data⁴. The nice thing about circuit models is that we have creative freedom of what we can put in the circuit diagram. We can create samples of varying intensities to see how the model will handle a simple circuit diagram to how it'll perform with a very dense circuit diagram which would have more circuit elements that would be harder for the object detector to pick up and push the limits of the model. Although, these examples that would be used for the model's assessment should be kept grounded and not be unrealistic to make the evaluation as fair as possible.

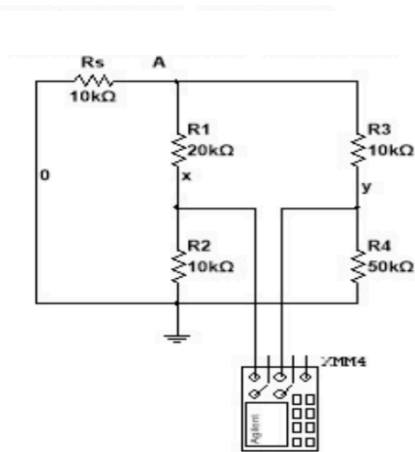
Model

Like mentioned from the proposal, we have many options for this project we can choose from². However, we decided to go for the YOLO platform as it is the most prevalent model and has had many iterations since its inception. As of now, you are able to use any of the YOLO 11 models from the ultralytics library. There are 5 different YOLO 11 models to choose from: YOLO11 (Detection), YOLO11-seg (Instance Segmentation), YOLO11-pose (Pose/Keypoints), YOLO11-obd (Oriented Detection), and YOLO11-cls (Classification)⁵. YOLO11-obd makes the most sense for our use case because our goal is to detect circuit elements that can be oriented in different 90 degree angles. There are also 5 different sizes to choose from: n, s, m, l, x. We will experiment with trying the small, medium, and large models to see which we should stick with for the project. Larger models inference better, but are slower to compute compared to smaller models.

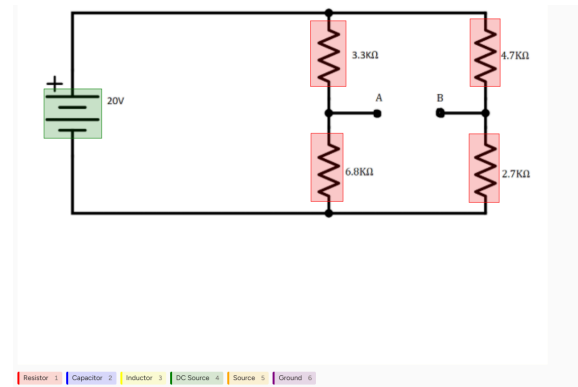
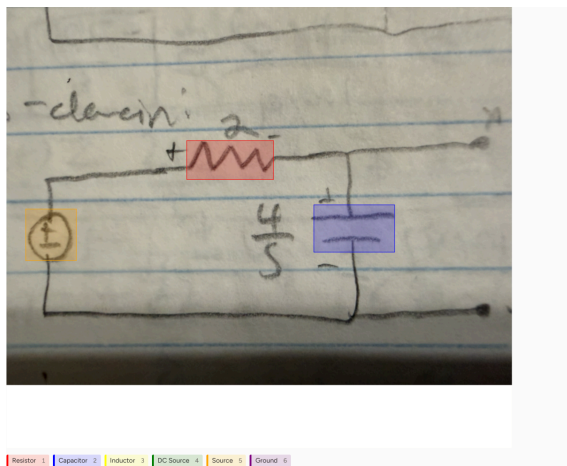
Dataset

This will be almost half of our project since we are creating this dataset from scratch and will try to configure it so it functions correctly when fed through the YOLO model. As mentioned earlier, we are using circuit models because of their flexibility and I also have lots of saved examples I can use from past notebooks, prior labs, and previous exams. After compiling all of the pictures, we have: 81 notebook photos, 106 lab circuit photos, and 13 online photos making a total of 200 circuit/circuit element photos for our dataset. Here are some examples:





We now need to prepare the dataset by creating labels for each of the photos. When looking at all the photos for the dataset, there are circuit elements that are very common, and there are elements that aren't as prevalent or are situational. Thus the 6 classes for each of the elements are: Resistor, Capacitor, Inductor, DC Voltage Source, Source (Circular DC/AC or other source), and Ground. There are a couple options for labeling, but Label Studio will be used for this project³. Label Studio allows you to upload all of your images, create the classes for the elements you want the model to detect, apply those labels, and export the dataset to a couple different formats. Here's what the labeled images looks like:



Upon exporting the labels, we have the option to choose YOLO-obb format which fits perfectly for the chosen model. After exporting the labels, it gives us a text doc of each pictures' label's bounding box. Although it didn't export the photos as the pictures folder was empty. They have a distinct hex number attached to the front of each label's name, so those needed to be removed to match the names of the pictures. I'm assuming if the pictures were exported successfully, they would also have the matching hex number. Here's an example of what a label file looks like:

```
0 0.20798514391829154 0.11513463324048283 0.2924791886350975 0.11513463324048283 0.2924791886350975 0.255338904363974
0.20798514391829154 0.255338904363974
0 0.24698235840297125 0.425183384811142 0.30362116991643456 0.426183384811142 0.30362116991643456 0.5718386406585237
0.24698235840297125 0.5718386406585237
2 0.24512534818941583 0.6666666666666665 0.35654596100278557 0.6666666666666665 0.35654596100278557
0.7353045496720211 0.24512534818941583 0.7353045496720211
2 0.5051067788072794 0.6833797585886723 0.6341689879294335 0.6833797585886723 0.6341689879294335 0.743732590529248
0.5051067788072794 0.743732590529248
2 0.48746518185849573 0.42422083779758596 0.59702878365831 0.42422083779758596 0.59702878365831 0.5273909006499535
0.48746518185849573 0.5273909006499535
1 0.9090064995357474 0.25998142989786444 1.0 0.25998142989786444 1.0 0.38811513463324043 0.9090064995357474
0.38811513463324043
```

The class number is the number all the way to the left which ranges from 0-5 and is used to identify the specific circuit element. Cap is 0, DC Source is 1, Ground is 2, Inductor is 3, Resistor is 4, and Source is 5. Now the dataset has its images and labels ready. I decided to use an 80/20 training/validation split since we will also have 6 pictures to evaluate performance of the model. The last thing to do is to create the yaml file which will show the paths of where the training and validation set are located and assign names to each of the class numbers.

Experiments and Results

Let's first establish which size model we should use. When running the trainer with an lr=0.001 and epochs=10 on the medium, our box loss rate was around 1.2 which when we would evaluate the test pictures with that loss rate, it would return a no predictions status. After increasing the amounts of epochs by

trying 25, 50, 125, the model was starting to detect the elements in the test images. Here's what the 200 epoch run looked like:

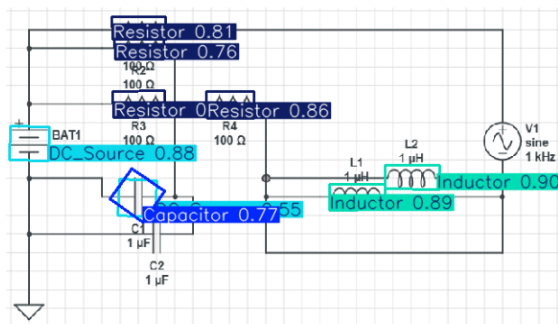
```
from ultralytics import YOLO

model = YOLO("yolo11m-obb.pt")

results = model.train(
    data = "/content/data.yaml",
    batch = 16,
    imgsz = 640,
    pretrained = True,
    epochs = 200,
    optimizer = "AdamW",
    lr0 = 0.001,
)
```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	Speed	Time	Acc
150/200	8.17G	0.352	0.268	1.889	41	640: 100%	10/10 1.711/s 5.8s	3/3 3.811/s 0.8s	0.999 0.728
Class	Images	Instances	Box(P)	R	mAP50 mAP50-95	100%			
all	48	84	0.817	0.876	0.899	0.728			
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	Speed	Time	Acc
200/200	8.22G	0.3499	0.2745	1.43	9	640: 100%	10/10 1.711/s 5.8s	3/3 4.611/s 0.7s	0.999 0.722
Class	Images	Instances	Box(P)	R	mAP50 mAP50-95	100%			
all	48	84	0.818	0.877	0.899	0.722			

It was able to detect all the circuit elements in the notebook pictures and computer pictures, but it missed the source symbol and capacitor symbol in the last test computer picture.



Training times were also reasonable as this run only took 30 minutes. Let's try the large model and see what we get.

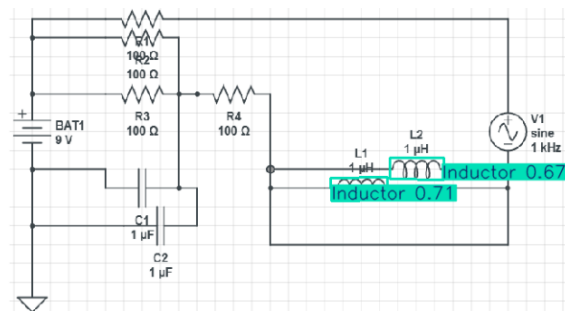
```
from ultralytics import YOLO

model = YOLO("yolo11l-obb.pt")

results = model.train(
    data = "/content/data.yaml",
    batch = 16,
    imgsz = 640,
    pretrained = True,
    epochs = 200,
    optimizer = "AdamW",
    lr0 = 0.001,
)
```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	Speed	Time	Acc
141/200	8.08G	0.392	0.284	1.9	19	640: 100%	10/10 1.711/s 7.5s	3/3 3.811/s 1.8s	0.999 0.722
Class	Images	Instances	Box(P)	R	mAP50 mAP50-95	100%			
all	48	84	0.817	0.876	0.899	0.728			

EarlyStopping: Training stopped early as no improvement observed in last 100 epochs. Best results observed at epoch 41, best model saved as best.pt. To update EarlyStopping(patience=100) pass a new patience value, i.e., 'patience=200' or use 'patience=0' to disable EarlyStopping.



Our results are way worse than the medium model. It wasn't able to detect most of the circuit elements, and the training times were worse. We would probably need a much larger dataset to use this model effectively. Now let's try the smaller model with the same parameters.

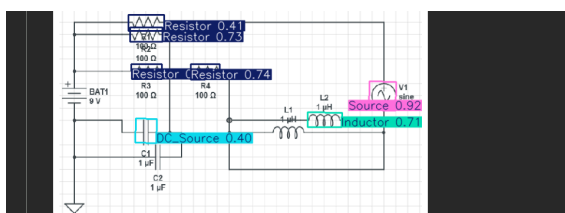
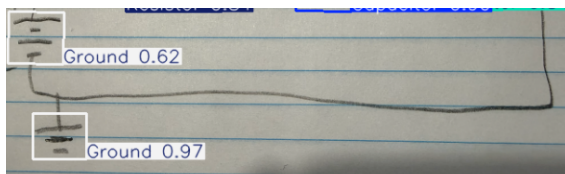
```
from ultralytics import YOLO

model = YOLO("yolo11s-obb.pt")

results = model.train(
    data = "/content/data.yaml",
    batch = 16,
    imgsz = 640,
    pretrained = True,
    epochs = 200,
    optimizer = "AdamW",
    lr0 = 0.001,
)
```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	Speed	Time	Acc
177/200	6.79G	0.392	0.284	1.9	19	640: 100%	10/10 2.611/s 3.5s	3/3 5.711/s 0.5s	0.999 0.722
Class	Images	Instances	Box(P)	R	mAP50 mAP50-95	100%			
all	48	84	0.817	0.876	0.899	0.728			

EarlyStopping: Training stopped early as no improvement observed in last 100 epochs. Best results observed at epoch 77, best model saved as best.pt. To update EarlyStopping(patience=100) pass a new patience value, i.e., 'patience=200' or use 'patience=0' to disable EarlyStopping.



It performed better than the large model, but we got a false positive as shown from the notebook first test image. We are also missing more elements in the last test photo and have lower confidence scores. Thus, we will choose to stay with the medium model, so let's further tune it.

Lets add a cosine learning rate since this is recommended to improve inferencing. Upon a Google search, it's best to stick with AdamW instead of trying something like NAdam since AdamW has better stability.

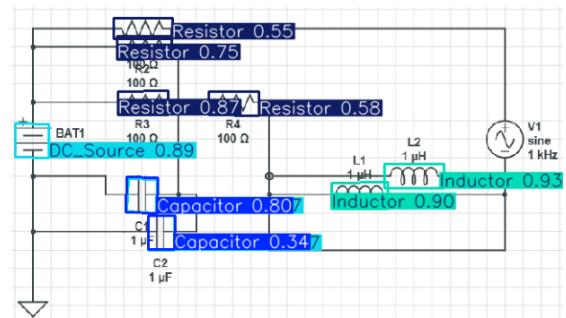
```
from ultralytics import YOLO

model = YOLO("yolo11m-obb.pt")

results = model.train(
    data = "/content/data.yaml",
    batch = 16,
    imgsz = 640,
    pretrained = True,
    epochs = 200,
    optimizer = "AdamW",
    lr0 = 0.001,
    cos_lr = True,
)
```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
200/200	8.21G	0.3623	0.3020	1.673	9	640: 100%
Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%
all	40	84	0.809	0.879	0.938	0.755

200 epochs completed in 0.438 hours.
 Optimizer stripped from /content/runs/obb/train/weights/last.pt, 42.40B
 Optimizer stripped from /content/runs/obb/train/weights/best.pt, 42.40B



This model performs better than the last model without cosine annealing. It seems like the trainer doesn't cut off for this model compared to the small and large models. Instead of increasing the number of epochs, let's increase the learning rate. The default final learning rate is 0.01. Let's increase the starting learning rate to be slightly higher than the final learning rate since the learning rate should be decaying to a smaller value instead of increasing ($lr_0=0.2$). However, when we do this our initial start is much worse than before.

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
50/200	8.21G	1.158	1.892	2.442	53	640: 100%
Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%
all	40	84	0.768	0.761	0.798	0.582

Let's try $lr_0 = 0.01$

```
results = model.train(
    data = "/content/data.yaml",
    batch = 16,
    imgsz = 640,
    pretrained = True,
    epochs = 200,
    optimizer = "AdamW",
    lr0 = 0.01,
    cos_lr = True,
)
```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
200/200	8.21G	0.573	0.3935	2.141	9	640: 100%
Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%
all	40	84	0.94	0.897	0.936	0.74

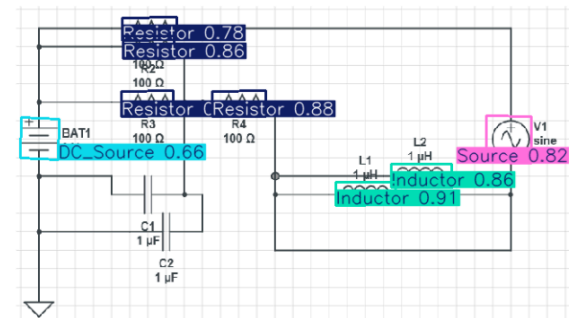
Again this lr is worse. Let's go back a decade but try a higher value: $lr_0 = 0.005$

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
177/200	8.18G	0.585	0.4285	2.495	26	640: 100%
Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%
all	40	84	0.939	0.899	0.937	0.755

EarlyStopping: Training stopped early as no improvement observed in last 100 epochs. Best results observed at epoch 77, best model saved as best.pt.
 To update EarlyStopping(patience=100) pass a new patience value, i.e. 'patience=100' or use 'patience=0' to disable EarlyStopping.

The model early stopped it with and it still performs worse than with $lr = 0.001$. Lastly, I want to try one more decade lower and see what happens.

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
200/200	8.21G	0.4668	0.4168	1.67	9	640: 100%
Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%
all	40	84	0.851	0.785	0.874	0.788



It didn't get the capacitors in the last test and overall performed better than last run, but not as good when we left $lr_0 = 0.001$. The default lr_0 value is 0.01, but we arbitrarily set the value to 0.001 initially as a baseline, but I guess it's the best value that works with this model for now, so let's leave $lr_0 = 0.001$. Now we should just use these parameters and just train for longer. Training times are still very short. The last run took around 25 minutes, so if the model has more room to learn, let's just increase the number of epochs. Let's train to 300 epochs and see what happens.

```
from ultralytics import YOLO

model = YOLO("yolo11m-obb.pt")

results = model.train(
    data = "/content/data.yaml",
    batch = 16,
    imgsz = 640,
    pretrained = True,
    epochs = 300,
    optimizer = "AdamW",
    lr0 = 0.001,
    cos_lr = True,
)
```

I noticed it keeps taking runs that are earlier than 200 epochs as the best run. I left the batch size at 16 for this entire time. Maybe a smaller batch size will give us better accuracy for our

model. Let's set the batch size to 8 and see what happens.

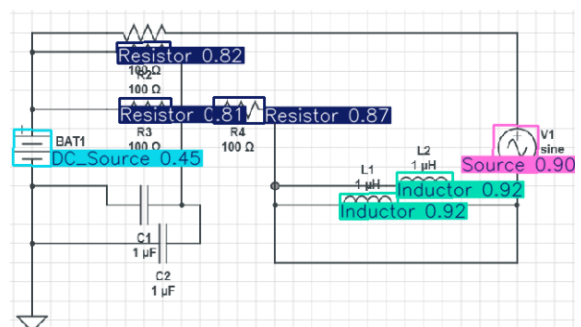
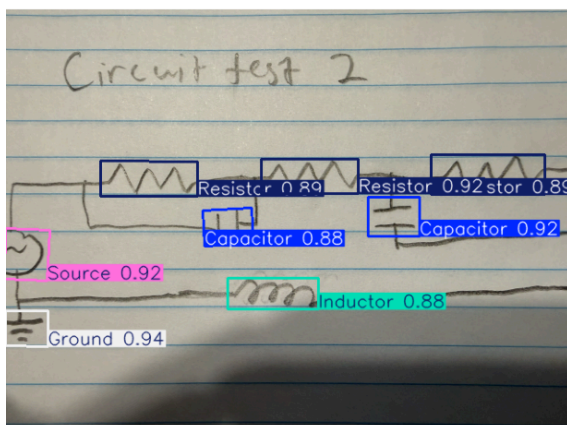
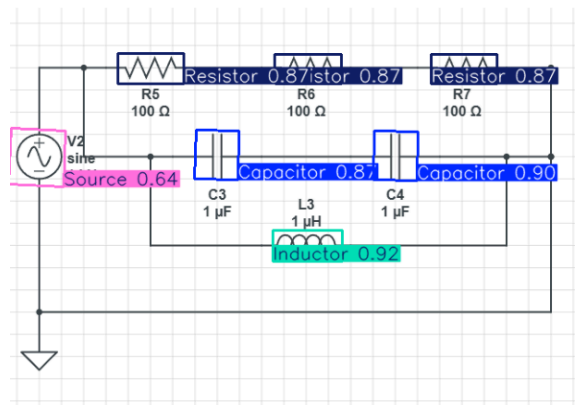
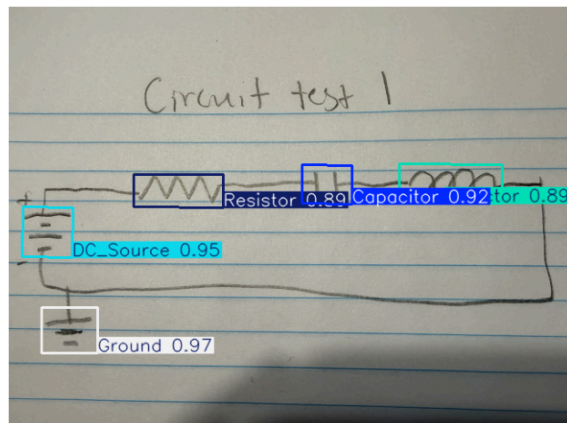
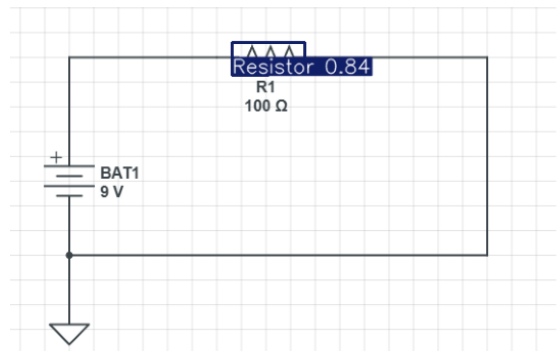
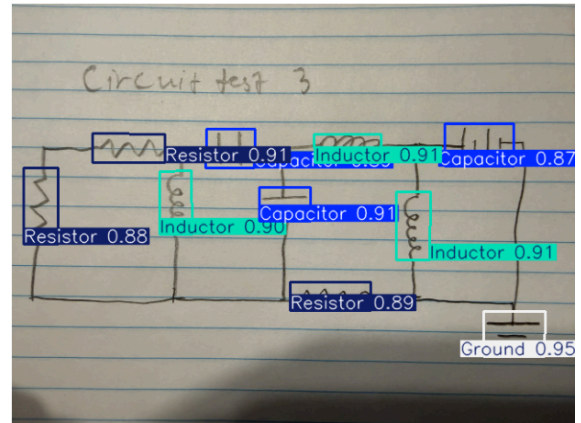
The results looked promising but it struggled to detect the computer test pictures. With more tuning from the learning rate, we can probably get better results, but we're low on time. For the final runs, we'll stick with these parameters as they gave us the best results so far. We need the patience parameter to stop the trainer from ending the training earlier.

```
from ultralytics import YOLO

model = YOLO("yolo11m-obb.pt")

results = model.train(
    data = "/content/data.yaml",
    batch = 16,
    imgsz = 640,
    pretrained = True,
    epochs = 200,
    optimizer = "AdamW",
    lr0 = 0.001,
    cos_lr = True,
    patience = 200,
)
```

```
Epoch: 200/200 | GPU mem: 8.75G | box loss: 0.3565 | cls loss: 0.284 | dfl loss: 1.776 | Instances: 11 | Size: 640 | 10/18 6.51t/s 1.5s |
Class: Images: Instances: Box(P): 8 | mAP50: mAP50-95: 100% | 3/3 12.21t/s 0.2s |
all: 48 | 84 | 0.967 | 0.76 | 0.923 | 0.777 |
200 epochs completed in 0.141 hours.
Optimizer stripped from /content/runs/obb/train2/weights/last.pt, 42.49B
Optimizer stripped from /content/runs/obb/train2/weights/best.pt, 42.49B
Validating /content/runs/obb/train2/weights/best.pt...
Ultralytics 8.3.240 Python 3.12.12 torch 2.9.0dev202 CUDA:0 (NVIDIA A100-SXM4-40GB, 48GB/761B)
YOLO11m-obb summary (fused): 124 Layers, 20,882,188 parameters, 0 gradients, 71.4 GB/1.8s
Ultralytics 8.3.240 Python 3.12.12 torch 2.9.0dev202 CUDA:0 (NVIDIA A100-SXM4-40GB, 48GB/761B)
YOLO11m-obb summary (fused): 124 Layers, 20,882,188 parameters, 0 gradients, 71.4 GB/1.8s
Class: Images: Instances: Box(P): 8 | mAP50: mAP50-95: 100% | 3/3 9.41t/s 0.3s |
all: 48 | 84 | 0.967 | 0.76 | 0.923 | 0.777 |
DC_Source: 4 | 0 | 0.989 | 0.667 | 0.913 | 0.765 |
Ground: 3 | 0.922 | 0.667 | 0.863 | 0.788 |
Inductor: 8 | 0 | 0.919 | 0.667 | 0.834 | 0.718 |
Resistor: 15 | 27 | 1 | 0.829 | 0.876 | 0.720 |
Source: 8 | 8 | 0.962 | 0.875 | 0.902 | 0.824 |
Speed: 0.28s preprocess, 1.44s inference, 0.06s loss, 1.66s postprocess per Image
Results saved to /content/runs/obb/train2
```



Using the last run to evaluate these circuits, we can see that it detects the notebook examples perfectly. When it comes to the computer circuit simulated examples, the results are mostly good, but not perfect.

Conclusion

This project feels like a demonstration of what is now possible with deep learning more so than anything. We did achieve the goals of: creating a dataset that is trainable for object detection, trained an object detection model to detect circuit elements from random examples, and optimized the training parameters for good results with a short training time of around 30 mins using the T4 and 7 mins using the A100. We can probably get better accuracy with this dataset by tuning the other hyperparameters with the smaller batch size, but due to time constraints, it was better to use the parameters determined earlier. It's actually really impressive how YOLO was able to achieve very good accuracy with such a small dataset. 200 examples is not a lot of data for an object detection algorithm to go off of. I thought the computer generated circuits would have perfect accuracy while the notebook examples would struggle, but I was wrong. Regardless, modern Deep Learning Object Detection models are able to be fine tuned with limited data in a short amount of time and produce great results. It just goes to show how far we've come just from a couple years ago compared to now. It'll be interesting to see how these later iteration models would perform on these training conditions.

References

- [1] M, T. (2025, November 11). *Key object detection metrics for computer vision*. Roboflow Blog.
<https://blog.roboflow.com/object-detection-metrics/>
- [2] GeeksforGeeks Writing Team. (2025, July 23). Object detection models. GeeksforGeeks.
<https://www.geeksforgeeks.org/computer-vision/object-detection-models/>
- [3] Skalski, P. (2025, February 3). *A simple way of creating a custom object detection model*. Medium.
<https://medium.com/data-science/chess-rolls-or-basketball-lets-create-a-custom-object-detection-model-ef53028eac7d>
- [4] Hemker, D., Maalouly, J., Mathis, H., Klos, R., & Ravanan, E. (2024, November 29). *From Schematics to Netlists – Electrical Circuit Analysis Using Deep-Learning Methods*. Advances in Radio Science.
<https://ars.copernicus.org/articles/22/61/2024/ars-22-61-20>
- [5] <https://docs.ultralytics.com/models/yolo11/#supported-tasks-and-modes>
- [6] <https://docs.ultralytics.com/usage/cfg/#train-settings>

Github and Dataset Links

- https://github.com/bwp7162/ECE_7123_Final_Project_Repo.git
- <https://www.kaggle.com/datasets/brycepoonai/dl-circuit-dataset-bp>
- https://colab.research.google.com/drive/1i6rEjniw4G0QsWC-rE_cKN622WWbb7R9?usp=sharing